Занятие 8: Работа с базой данных

Подключение, драйверы, SQL, ORM

Слайд 1: Путь данных от HTTP до БД

Полный цикл запроса:

```
1. HTTP Request
   ↓
2. Web Server (Nginx)
   ↓
3. Application (Python/Flask)
   ↓
4. Database Driver (psycopg2/mysql-connector)
   ↓
5. Database Server (PostgreSQL/MySQL)
   ↓
6. Response back through the chain
```

Ключевые компоненты:

- **Application** - бизнес-логика
- **Driver** - библиотека для связи с БД
- **Database Server** - хранение данных

Слайд 2: Подключение к БД

Параметры подключения:

- **Host** - адрес сервера БД
- **Port** - порт (PostgreSQL: 5432, MySQL: 3306)
- **Database** - имя базы данных
- **User** - имя пользователя
- **Password** - пароль

Пример строки подключения:

```
postgresql://user:password@localhost:5432/mydb
mysql://user:password@localhost:3306/mydb
```

Пример на Python (psycopg2):

```python
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    port=5432,
    database="mydb",
    user="postgres",
    password="password"
)
```

Слайд 3: Драйверы БД

Что такое драйвер?

Драйвер - библиотека, которая позволяет приложению общаться с БД.

Популярные драйверы:

БД	Python драйвер	Node.js драйвер
PostgreSQL	psycopg2, asyncpg	pg
MySQL	mysql-connector-python, PyMySQL	mysql2
SQLite	sqlite3 (встроенный)	better-sqlite3

Пример установки:

```
pip install psycopg2-binary
pip install mysql-connector-python
```

Слайд 4: Выполнение SQL запросов

Шаги выполнения запроса:

1. Создать подключение (Connection)
2. Создать курсор (Cursor)
3. Выполнить запрос (execute)
4. Получить результат (fetch)
5. Закрыть курсор и подключение

Пример:

```
import psycopg2

conn = psycopg2.connect("...")
cursor = conn.cursor()

cursor.execute("SELECT * FROM users")
results = cursor.fetchall()

cursor.close()
conn.close()
```

Слайд 5: INSERT - вставка данных

Пример вставки пользователя:

```
cursor.execute(
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)",
    ("John Doe", "john@example.com", 30)
)
conn.commit()
```

Важно:

- Используйте параметризованные запросы (%s или ?)
- Не подставляйте значения напрямую (SQL injection!)
- Вызовите commit() для сохранения изменений

Плохо (SQL injection):

```
name = input("Name: ")
cursor.execute(f"INSERT INTO users (name) VALUES ('{name}')")  # НЕ ДЕЛАЙТЕ ТАК!
```

Слайд 6: SELECT - чтение данных

Примеры запросов:

```
# Все записи
cursor.execute("SELECT * FROM users")
users = cursor.fetchall()
# [(1, 'John', 'john@example.com', 30), ...]

# Одна запись
cursor.execute("SELECT * FROM users WHERE id = %s", (1,))
user = cursor.fetchone()
# (1, 'John', 'john@example.com', 30)

# Определённые колонки
cursor.execute("SELECT name, email FROM users")
names = cursor.fetchall()
```

fetchone() vs fetchall():

- `fetchone()` - одна запись (или None)
- `fetchall()` - все записи (список)
- `fetchmany(n)` - n записей

Слайд 7: UPDATE и DELETE

UPDATE - обновление:

```
cursor.execute(
    "UPDATE users SET age = %s WHERE id = %s",
    (31, 1)
)
conn.commit()
```

DELETE - удаление:

```
cursor.execute("DELETE FROM users WHERE id = %s", (1,))
conn.commit()
```

Важно:

- Всегда используйте WHERE
- Проверяйте количество затронутых строк
- Используйте транзакции для связанных изменений

Проверка затронутых строк:

```
cursor.execute("UPDATE users SET age = 30 WHERE id = %s", (1,))
print(f"Updated {cursor.rowcount} rows")
```

Слайд 8: Обработка ошибок

Типичные ошибки:

```
try:
    conn = psycopg2.connect("...")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users")
    results = cursor.fetchall()

except psycopg2.OperationalError as e:
    print(f"Connection error: {e}")

except psycopg2.ProgrammingError as e:
    print(f"SQL syntax error: {e}")

except Exception as e:
    print(f"Unexpected error: {e}")

finally:
    if 'cursor' in locals():
        cursor.close()
    if 'conn' in locals():
        conn.close()
```

Используйте context manager:

```
with psycopg2.connect("...") as conn:
    with conn.cursor() as cursor:
        cursor.execute("SELECT * FROM users")
        results = cursor.fetchall()
# Автоматически закроет cursor и conn
```

Слайд 9: SQL Injection

Что такое SQL Injection?

Атака, при которой злоумышленник внедряет SQL код в запрос.

Пример уязвимости:

```
email = input("Email: ")
query = f"SELECT * FROM users WHERE email = '{email}'"
cursor.execute(query)
```

Атака:

```
Email: ' OR '1'='1
Query: SELECT * FROM users WHERE email = '' OR '1'='1'
Результат: Все пользователи!
```

Защита - параметризованные запросы:

```
email = input("Email: ")
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Слайд 10: Параметризованные запросы

Почему это безопасно?

Драйвер экранирует параметры отдельно от SQL:

```
# Безопасно
cursor.execute(
    "SELECT * FROM users WHERE email = %s",
    ("' OR '1'='1",)
)
# Будет искать пользователя с email "' OR '1'='1"
```

Разные синтаксисы:

Драйвер	Синтаксис
psycopg2	%s
mysql-connector	%s
sqlite3	?
asyncpg	$1, $2

Пример с несколькими параметрами:

```
cursor.execute(
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)",
    ("John", "john@example.com", 30)
)
```

Слайд 11: JOIN - объединение таблиц

Пример схемы:

```
users:      orders:
id          id
name        user_id
email       product
            total
```

INNER JOIN:

```
cursor.execute("""
    SELECT users.name, orders.product, orders.total
    FROM users
    INNER JOIN orders ON users.id = orders.user_id
""")
```

LEFT JOIN:

```
cursor.execute("""
    SELECT users.name, orders.product
    FROM users
    LEFT JOIN orders ON users.id = orders.user_id
""")
# Вернёт всех пользователей, даже без заказов
```

Слайд 12: Миграции

Что такое миграции?

Миграции - управляемые изменения схемы БД.

Зачем нужны:

- Версионирование схемы БД
- Применение изменений в команде
- Откат изменений

Пример миграции:

```sql
-- migration_001_create_users.sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    age INTEGER CHECK (age >= 18),
    created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
-- migration_002_add_phone.sql
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
```

Слайд 13: Инструменты миграций

Популярные инструменты:

Alembic (Python):

```
pip install alembic
alembic init alembic
alembic revision --autogenerate -m "Add users table"
alembic upgrade head
alembic downgrade -1
```

Flyway (Java/Go/Python):

```
-- V1__Create_users_table.sql
CREATE TABLE users (...);

-- V2__Add_phone_column.sql
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
```

Пример Alembic:

```
def upgrade():
    op.create_table('users',
        sa.Column('id', sa.Integer(), primary_key=True),
        sa.Column('name', sa.String(50)),
        sa.Column('email', sa.String(100))
    )

def downgrade():
    op.drop_table('users')
```

Слайд 14: ORM - что это?

ORM (Object-Relational Mapping)

Преобразование объектов в записи БД и наоборот.

Пример без ORM:

```
cursor.execute("SELECT * FROM users WHERE id = %s", (1,))
row = cursor.fetchone()
user = {'id': row[0], 'name': row[1], 'email': row[2]}
```

Пример с ORM (SQLAlchemy):

```
user = session.query(User).filter_by(id=1).first()
print(user.name)
```

Преимущества ORM:

- Работаем с объектами, а не SQL
- Автоматическая генерация SQL
- Защита от SQL injection
- Миграции через модели

Слайд 15: SQLAlchemy пример

Определение модели:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

Base = declarative_base()

class User(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    name = Column(String(50))
    email = Column(String(100), unique=True)
    age = Column(Integer)

# Создание таблицы
engine = create_engine('postgresql://...')
Base.metadata.create_all(engine)

Session = sessionmaker(bind=engine)
session = Session()
```

CRUD операции:

```
# Create
user = User(name="John", email="john@example.com", age=30)
session.add(user)
session.commit()

# Read
user = session.query(User).filter_by(id=1).first()
```

Слайд 16: Connection Pooling

Что такое connection pool?

Пул подключений - кэш открытых подключений к БД.

Зачем нужен:

- Создание подключения - дорого
- Переиспользование подключений
- Ограничение количества подключений

Пример с SQLAlchemy:

```
from sqlalchemy.pool import QueuePool

engine = create_engine(
    'postgresql://...',
    poolclass=QueuePool,
    pool_size=5,
    max_overflow=10
)
```

Без пула:

```
Request 1: Create connection → Query → Close
Request 2: Create connection → Query → Close
# Медленно!
```


С пулом:

Слайд 17: Асинхронный доступ к БД

Синхронный vs Асинхронный:

```
# Синхронный (psycopg2)
def get_user(user_id):
    conn = psycopg2.connect("...")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
    return cursor.fetchone()
# Блокирует поток

# Асинхронный (asyncpg)
async def get_user(user_id):
    conn = await asyncpg.connect("...")
    row = await conn.fetchrow("SELECT * FROM users WHERE id = $1", user_id)
    await conn.close()
    return row
# Не блокирует
```

Когда использовать асинхронный:

- Высокая нагрузка
- Много одновременных запросов
- FastAPI, aiohttp

Слайд 18: Пример с asyncpg

```python
import asyncpg
import asyncio

async def main():
    # Подключение
    conn = await asyncpg.connect(
        host='localhost',
        port=5432,
        database='mydb',
        user='postgres',
        password='password'
    )

    # INSERT
    await conn.execute(
        "INSERT INTO users (name, email) VALUES ($1, $2)",
        "John", "john@example.com"
    )

    # SELECT
    user = await conn.fetchrow(
        "SELECT * FROM users WHERE id = $1",
        1
    )
    print(user)

    # Закрытие
    await conn.close()

asyncio.run(main())
```

Слайд 19: Практическое задание

Задание 1: Подключение к БД

- Создать подключение к PostgreSQL/MySQL
- Выполнить простой SELECT запрос
- Обработать ошибки подключения

Задание 2: CRUD операции

- Реализовать INSERT, SELECT, UPDATE, DELETE
- Использовать параметризованные запросы
- Проверить количество затронутых строк

Задание 3: JOIN

- Создать две связанные таблицы
- Выполнить INNER JOIN и LEFT JOIN
- Объяснить разницу

Задание 4: Миграции

- Создать миграцию для таблицы
- Применить миграцию
- Создать миграцию для изменения
- Откатить миграцию

Слайд 20: Вопрос для собеседования

Вопрос: Объясните безопасную запись и чтение данных из БД.

Ожидаемый ответ:

1. Использовать параметризованные запросы для защиты от SQL injection
2. Использовать транзакции для атомарности операций
3. Обрабатывать ошибки (connection error, syntax error)
4. Использовать connection pooling для производительности
5. Закрывать подключения (или использовать context manager)
6. Валидировать данные перед записью
7. Использовать миграции для управления схемой